

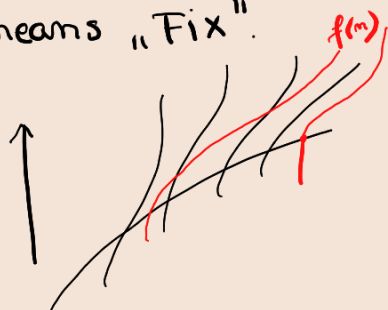
1) Def: A problem = a finite set of input and output variables.

$$P = \{x_i = \overline{0, m-1} \in D_{x_i}\}$$

2) Brute force: the algorithm that solves any problem.

```
for  $\alpha_0$  in  $D_{x_0}$  :  $|D_{x_0}|$ 
  for  $\alpha_1$  in  $D_{x_1}$  :  $|D_{x_1}|$ 
    ...
    for  $\alpha_{k-1}$  in  $D_{x_{m-1}}$  :  $|D_{x_{m-1}}|$ 
      candidate =  $(\alpha_0, \alpha_1, \dots, \alpha_{m-1})$ 
      if is_solution(candidate)
```

3) Θ means "Fix".



Complexitate bună: când f crește încet.

$\Omega(f(m))$ = totalitatea funcțiilor pe care le-am desenat

→ ele cresc mai rapid la ∞

→ ele cresc mai rapid sau la fel cu $f(m)$

" $\geq$ "

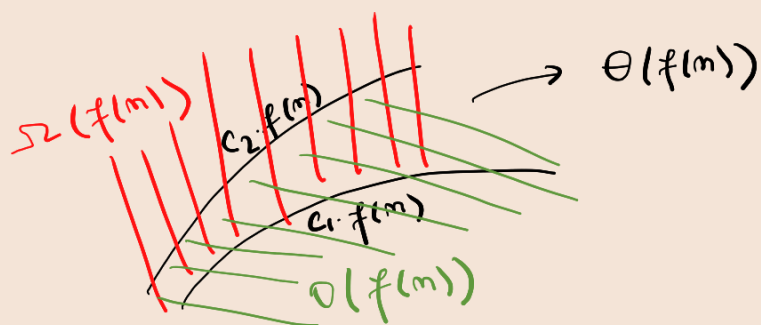
$O(f(m))$ = totalitatea funcțiilor care cresc mai încet la ∞

→ ele cresc mai încet sau la fel cu $f(m)$

→ "se poate mai bine"

" $\leq$ "

4) Înmulțim cu o constantă: $c_2 > c_1 > 0$ (scalari / constante)



$$\Omega(f(m)) \cap O(f(m)) = \Theta(f(m)) \text{ "Fix"}$$

5) $N_0, N_1, N_2, N_3 \Rightarrow \underbrace{|D_{N_0}|}_{10} \times \underbrace{|D_{N_1}|}_{10} \times \underbrace{|D_{N_2}|}_{10} \times \underbrace{|D_{N_3}|}_{10} = 10^4$
 $\in \{0, \dots, 9\}$

6) Eficientizare:

candidate = $\emptyset$

for α_0 in D_{r_0} :

candidate.append(α_0)

for α_1 in D_{r_1} :

candidate.append(α_1)

...

for α_{k-1} in $D_{r_{k-1}}$:

candidate.append(α_{k-1})

if is-solution(candidate)

...

pop()

pop()

pop()

if is-consistent(candidate) $\Rightarrow$ DIVIDE ET IMPERA
 BACKTRACKING $\begin{cases} 50\% T \\ 50\% F \end{cases}$

mai puțini pași

$O(f(m))$

$\hookrightarrow$ isn't that supposed to be the worst case?

pe măsură ce problema devine mai mare, algoritmul nu devine mult mai lent $\Rightarrow$ asta înseamnă complexitate bună

↳ Tabel: bag în listă cu append (lipsește)

→ Append: pune în coada unei liste o valoare

→ No more Θ , but O .

{ Backtracking is as efficient as you make it.
 Not backtracking, but brute force is inefficient.

Θ backtracking $\rightarrow O$
 (initial complexity)

În cel mai rău caz, e $\leq$ deci mai rapid sau la fel de rapid (crește mai încet sau la fel de încet)

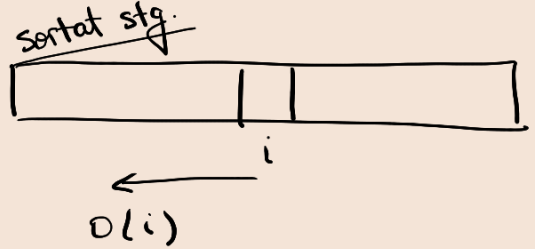
7) Selection sort works on brute force: $\Theta(m^2) = \frac{m(m+1)}{2}$

Bubble sort $\rightarrow$ alg. de sortare cu $O(m^2)$ și care folosește backtrack.

Insertion sort

↳ tot ce e în stg. e sortat

$$\sum_{i=0}^{n-1} O(i) = O\left(\sum_{i=0}^{n-1} i\right) = O(n^2)$$



$\Theta(n \cdot \log_2 n)$ - Merge Sort

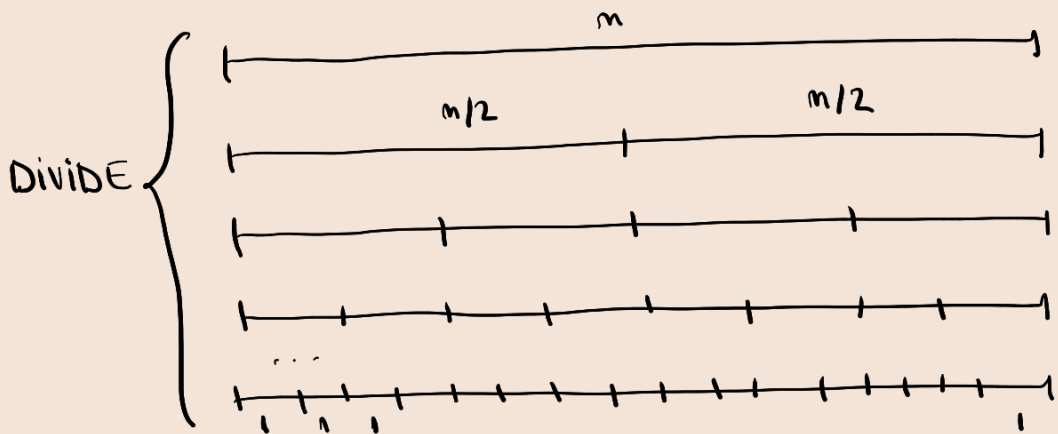
$O(n \cdot \log_2 n)$ - Heap Sort
- Quick Sort

8) DIVIDE ET IMPERA

k pasi

$$\Rightarrow \frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow k = \log_2 n.$$

↳ impart la 2 de k ori.



→ liste multe cu un singur element $\Rightarrow$
 $\Rightarrow$ sortate deja.

